

System Architecture Document

AI-Powered After-Call Automation Platform

Document Type: System Architecture Specification

Prepared by: Senior Solution Architect

Version: 1.0

1. System Overview

The AI-Powered After-Call Automation Platform is designed to automate the work that a customer service or field agent normally performs *after* a call ends — reviewing what was discussed, looking up the customer, finding relevant policy or product documentation, extracting the correct structured data, validating that data against business rules, deciding what should happen next, and storing a final, auditable record.

The system ingests a call transcript (voice-to-text or already-transcribed), grounds every extracted fact strictly in that transcript, retrieves supporting documentation *before* any generation step, applies deterministic business rules in code (not in prompts), and routes the outcome through one of three paths: **Auto-Save**, **Human Review**, or **Escalation**.

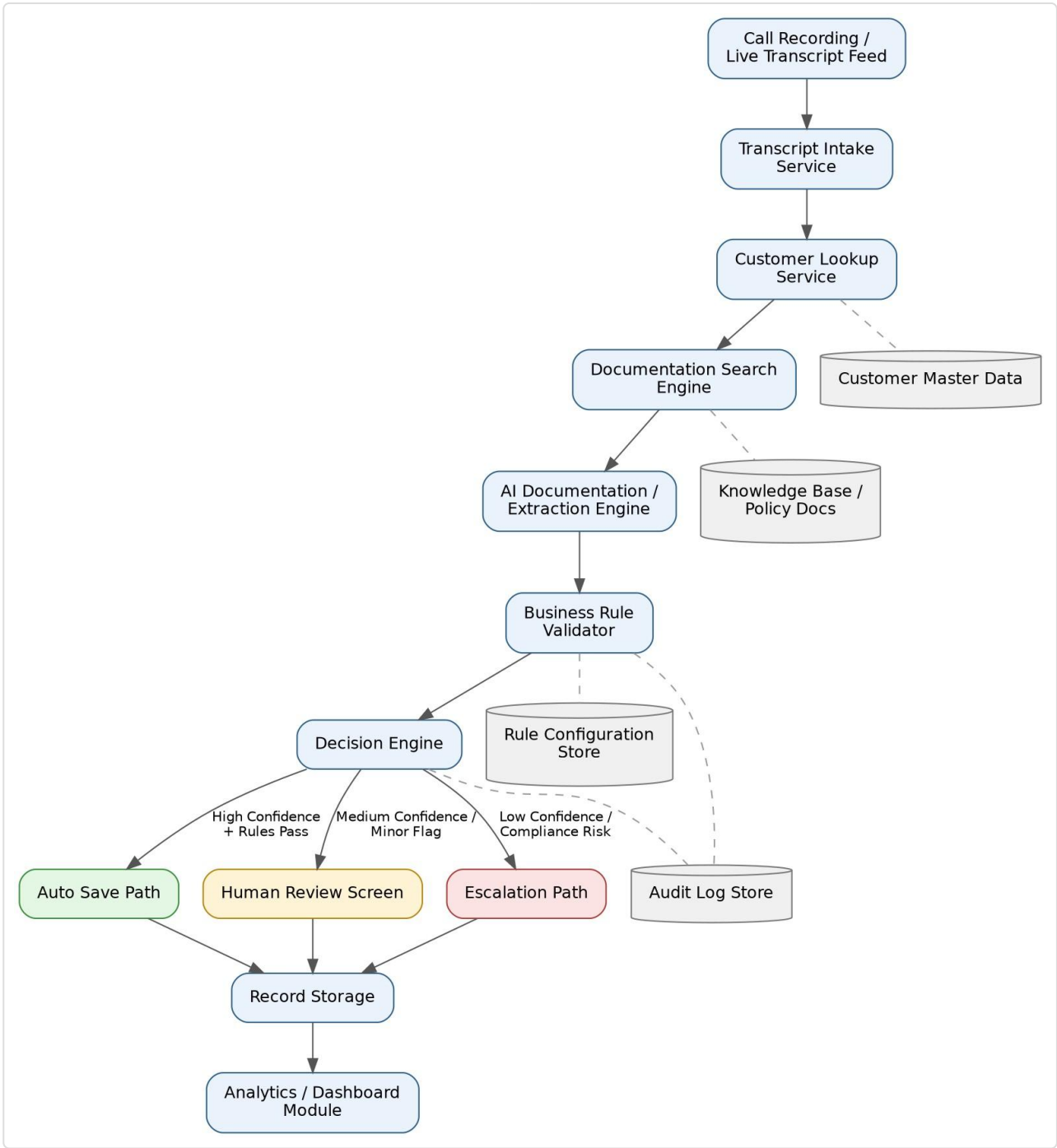
The platform is not a "transcribing and summarize" tool. It is a **decision-grade automation system**: its output is trusted enough to update systems of record, and therefore its architecture is built around grounding, verification, and auditability rather than raw generative fluency.

2. Architecture Goals

Goal	Description
Grounding over generation	Every field in the final record must be traceable to an explicit span in the transcript. Nothing is invented.
Retrieval before generation	Documentation and policy context are fetched <i>before</i> the AI extraction step, never fabricated post-hoc.
Deterministic business logic	Business rules (thresholds, eligibility, compliance checks) live in code as explicit, testable, versioned logic — never as prompt instructions the model might ignore or misinterpret.
Human-in-the-loop by design	Low-confidence or high-risk outcomes are routed to a reviewer; the system never silently guesses on consequential decisions.

Goal	Description
Auditability	Every decision (auto-save, review, escalate) has a recorded reason, confidence score, and rule trace.
Modularity	Each stage (lookup, search, extraction, validation, decisioning) is an independently deployable, independently testable service.
Scalability	The pipeline must handle bursty call volumes (e.g., end-of-shift spikes) without backpressure on upstream systems.
Extensibility	New document types, new business rules, and new decision categories can be added without re-architecting the pipeline.

3. High-Level Architecture Diagram



Reading the diagram: Data flows strictly left-to-right/top-to-bottom through the pipeline. Each stage only receives the outputs of the prior stage plus its own supporting data store — no stage reaches "ahead" in the pipeline, which is what enforces the retrieve-before-generate and validate-before-decide guarantees described later in this document.

4. Component Descriptions

4.1 Transcript Intake

The entry point of the system. Accepts finished call transcripts from the telephony/CCaaS platform (or a speech-to-text service upstream) along with call metadata (agent ID, call timestamp, call duration, channel, customer-provided identifiers if any). Responsibilities:

- Normalizes transcript format (speaker-labeled turns, timestamps per utterance).
- Performs light validation (minimum length, language detection, corrupt-audio-to-text rejection).
- Assigns a unique **Call Record ID** that follows the record through every downstream stage.
- Emits the normalized transcript onto the internal event stream for the next stage.

Intake does **no interpretation** — it is a structural, not semantic, gatekeeper.

4.2 Customer Lookup Service

Resolves the caller's identity against the customer master data store using whatever identifiers are available (phone number, account number, spoken name + verification answers captured in the transcript). Responsibilities:

- Queries CRM/customer master systems via API.
- Returns a **Customer Context Object**: account status, product holdings, tenure, prior case history.
- Flags "no confident match" explicitly rather than guessing — this flag alone can be enough to route a call to Human Review later.

This component runs early because every downstream stage (documentation relevance, rule validation, decisioning) depends on knowing *who* the customer is and *what they are entitled to*.

4.3 Documentation Search Engine

A retrieval component (not a generative one) that searches the knowledge base — product policies, compliance scripts, SOPs, pricing tables — for content relevant to the transcript's topic and the resolved customer's product context. Responsibilities:

- Converts transcript intent (e.g., "cancellation request", "billing dispute") into search queries.
- Executes retrieval (semantic + keyword) against the knowledge base.
- Returns a ranked set of **source-tagged documentation snippets**, each with a document ID, version, and effective date.

This step exists specifically to prevent hallucination: the AI Documentation Engine is never allowed to answer from parametric memory about policy — it is only allowed to work from what this component hands it.

4.4 AI Documentation Engine (AI Extraction)

The generative component of the pipeline, but a tightly constrained one.

It receives three inputs only:

- (1) the transcript,
- (2) the Customer Context Object,
- (3) the retrieved documentation snippets.

Responsibilities: -

- Extracts structured fields (e.g., "reason for call," "resolution offered," "amount discussed," "follow-up required") strictly from the transcript.
- For every extracted field, attaches a **transcript span reference** (which line/turn supports this value) and a **confidence score**.
- Where a field depends on a policy interpretation (e.g., "is this refund within policy"), it cites the specific retrieved snippet used — it does not answer from general knowledge. - Produces no business decisions, it produces *facts and citations only*.

4.5 Business Rule Validator

A pure, deterministic, code-based rule engine — explicitly **not** part of the AI model.

Responsibilities:

- Loads versioned business rules from the Rule Configuration Store (e.g., refund limits, mandatory disclosures, regulatory thresholds).
- Validates every AI-extracted field against these rules.

Produces a structured validation result:

pass, fail, or "needs-judgment," each with a machine-readable reason code.

Because this logic lives in code rather than in a prompt, it is unit-testable, version-controlled, and cannot be "talked around" by unusual transcript phrasing the way a prompt-based rule could be.

4.6 Decision Engine

Consumes the AI extraction output, its confidence scores, and the Business Rule Validator's results to determine the record's disposition.

Responsibilities:

- Applies decision thresholds (see Section 9) to select **Auto-Save**, **Human Review**, or **Escalation**.
- Writes a decision trace: which rule or confidence threshold drove the outcome. - Hands the record and its trace to the appropriate downstream path.

4.7 Human Review Screen

The interface where a human reviewer inspects records flagged for review.

Responsibilities:

- Displays the transcript, the AI-extracted fields side-by-side with their transcript citations, the retrieved documentation used, and the rule validator's flags.
- Allows accept, edit, or reject of individual fields (not just the whole record).
- Every edit is logged into a correction event, which feeds back into quality monitoring and model evaluation.

4.8 Record Storage

The system of records for finalized after-call records. Responsibilities: - Persists the final structured record, the original transcript, the documentation citations, the rule validation results, and the full decision trace as one immutable, versioned unit. - Supports append-only audit history (no destructive overwrites). - Exposes query APIs for downstream consumers (analytics, compliance export, CRM sync-back).

4.9 Analytics Module

Reads (never writes to) Record Storage to produce operational and quality insight. Responsibilities: Tracks auto-save rate, review rate, escalation rate over time. - Tracks reviewer correction patterns (which fields get corrected most, by which rule, by which agent/team). - Feeds a continuous improvement loop for both the extraction model and the business rule set.

5. End-to-End Workflow

1. A call ends; the transcript arrives at **Transcript Intake**, which validates and normalizes it and assigns a Call Record ID.
 2. **Customer Lookup** resolves the caller's identity and account context.
 3. **Documentation Search** retrieves the policy/product documentation relevant to the call's topic and the customer's product holdings.
 4. **AI Documentation Engine** extracts structured, transcript-grounded fields, citing both transcript spans and retrieved documents.
 5. **Business Rule Validator** checks every field against deterministic, versioned business rules.
 6. **Decision Engine** combines confidence scores and rule outcomes to choose Auto-Save, Review, or Escalation.
 7. If routed to **Human Review**, a reviewer accepts/edits/rejects fields; corrections are logged.
 8. The final record — whichever path it took — is written to **Record Storage** with a full audit trail.
 9. **Analytics Module** continuously reads storage to report on volumes, quality, and correction trends.
-

6. Data Flow

The system is built as an **event-driven pipeline**: each stage publishes its output as an event that the next stage consumes, rather than one monolithic synchronous call chain. This allows each stage to be scaled and retried independently.

```
Transcript Event
  → Customer Context Event (Lookup output + original transcript)
    → Documentation Bundle Event (retrieved snippets + prior payload)
      → Extraction Event (structured fields + citations + prior payload)
        → Validation Event (rule results + prior payload)
          → Decision Event (routing outcome + full trace + prior
payload)
            → Storage Write (final immutable record)
```

Each event carries **its own payload plus everything accumulated before it**, so any stage — and any later auditor — can reconstruct the full "why" behind a field without needing to query five different systems.

7. AI Processing Pipeline

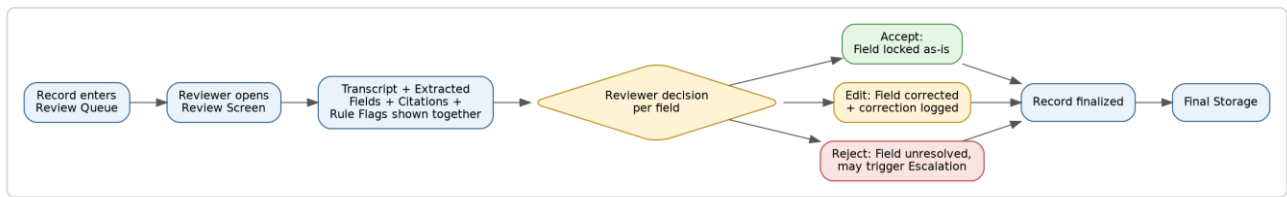
The AI Documentation Engine is deliberately the *only* generative step in the pipeline, and it is boxed in on both sides:

- **Before it:** Customer Lookup and Documentation Search have already narrowed the world of facts it is allowed to draw from — the customer's actual account context and the actual, versioned policy text.
- **After it:** the Business Rule Validator re-checks every output against code-based rules, so the model's output is treated as a *proposal*, not a final answer.

Internally, the extraction step works field-by-field: 1. Identify candidate fields required by the record schema for this call type. 2. For each field, locate supporting transcript span(s). 3. If the field requires policy interpretation, attach the specific retrieved documentation snippet used. 4. Assign a confidence score per field based on clarity and directness of transcript evidence. 5. Flag any field with no clear transcript support as "unsupported" rather than inferring a plausible value.

This field-level (not record-level) confidence scoring is what allows the Decision Engine to send a record to review for *one weak field* while still trusting the rest.

8. Human Review Flow



The Human Review Screen is intentionally field-granular rather than record-granular: a reviewer is never forced to accept or reject an entire record as one unit. This keeps human effort focused only on the specific fields the system itself flagged as uncertain, while still allowing the reviewer to override anything.

Every correction is logged with the original AI value, the corrected value, the reviewer ID, and a timestamp — this log is the primary input to model and rule-set improvement over time.

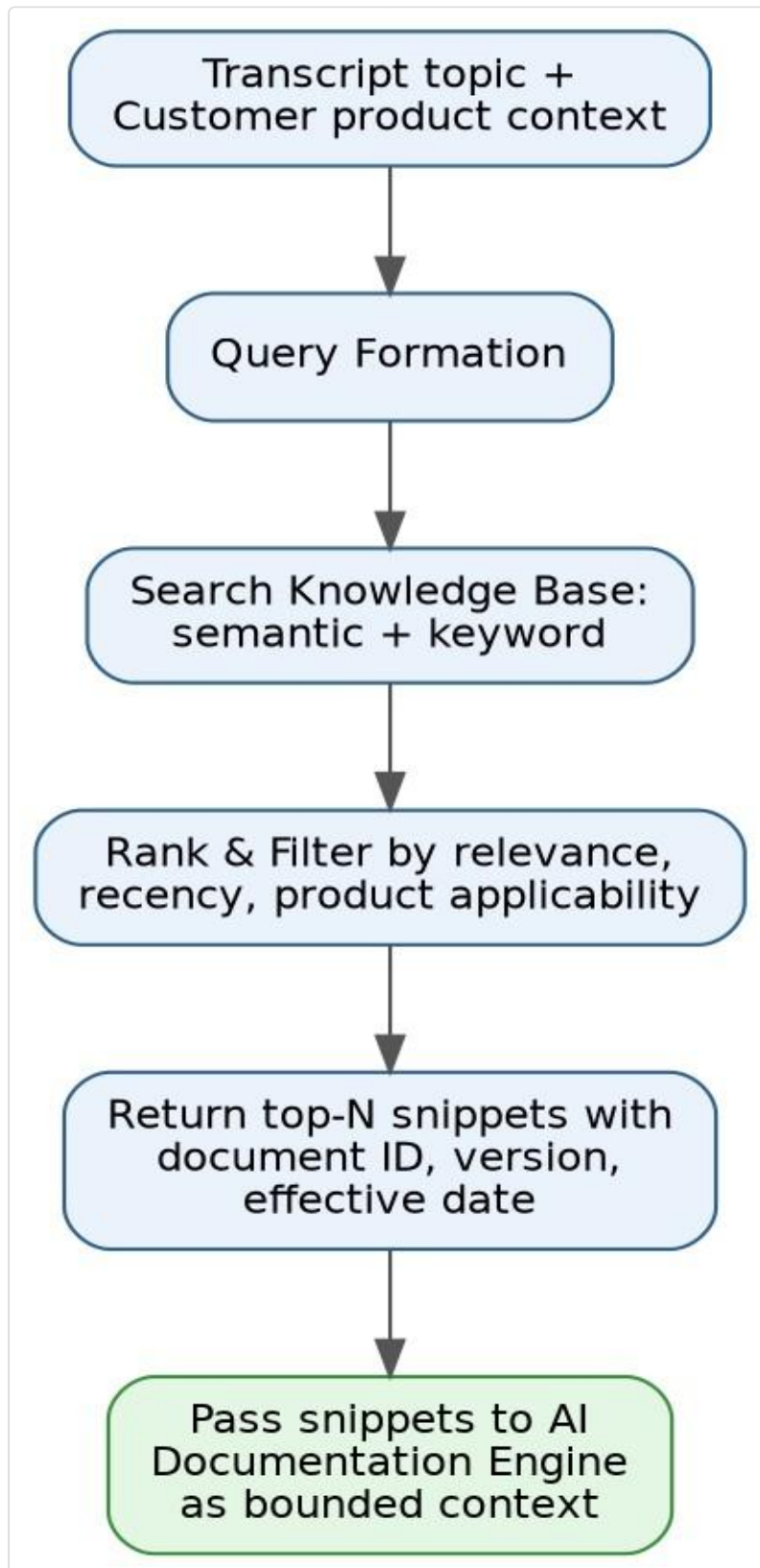
9. Business Decision Engine

The Decision Engine encodes routing logic in code, using two inputs from the previous stages: **field level confidence scores** from the AI Documentation Engine, and **rule outcomes** from the Business Rule Validator. It never makes a judgment call itself — it applies explicit, versioned decision policy.

Condition	Outcome
All required fields above confidence threshold and all business rules pass	Auto-Save
One or more fields below confidence threshold, but no business rule failure	Human Review
Any business rule fails, or a compliance-sensitive rule is triggered, or customer lookup was unresolved	Escalation
Reviewer rejects a field during Human Review	Escalation (rerouted)

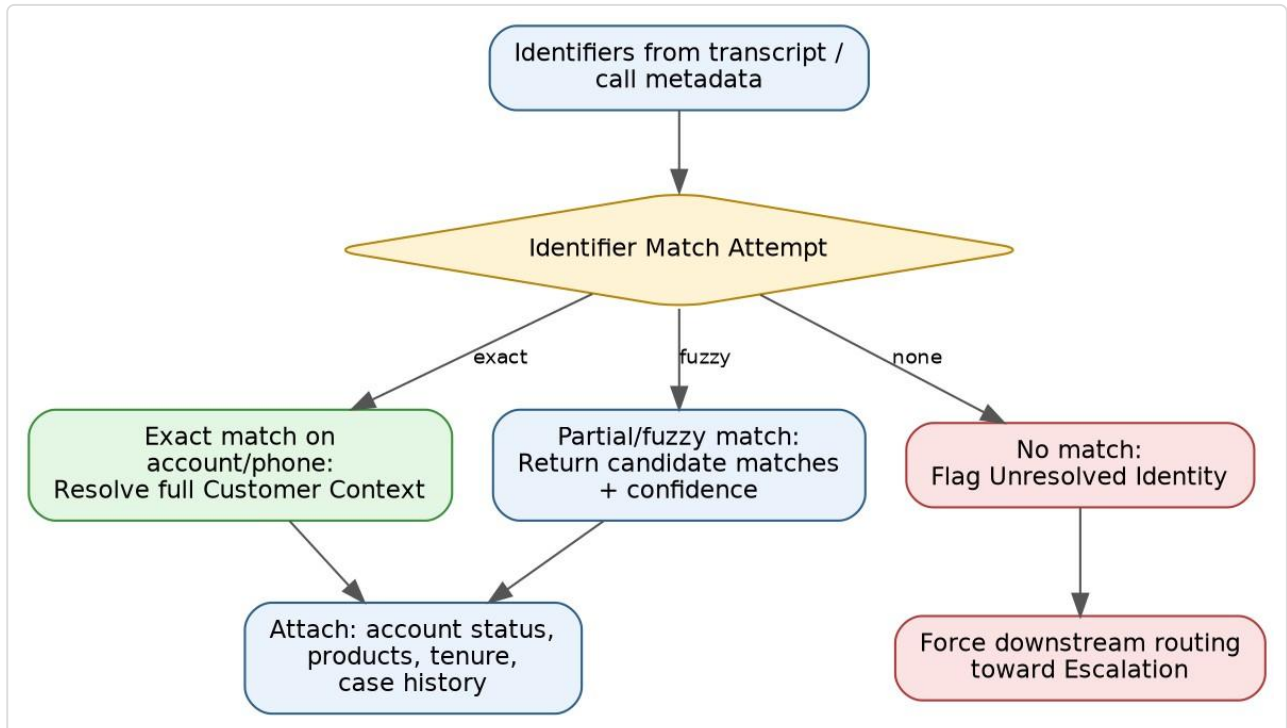
Because thresholds and rule-to-outcome mappings are stored as versioned configuration rather than hardcoded constants, they can be tuned per call type, per product line, or per regulatory jurisdiction without redeploying the pipeline.

10. Knowledge Retrieval Flow



The retrieval step is versioned and timestamped deliberately: policy documents change over time, and a call handled under an old policy must be evaluated against the policy version that was actually in effect at that time. The Documentation Search Engine always resolves to the effective-dated version of a document relative to the call's timestamp, not the current live version.

11. Customer Lookup Flow



An unresolved or low-confidence customer match is treated as a first-class signal — it is passed forward explicitly rather than silently defaulted, because every downstream stage (especially business rule validation) depends on knowing the customer's actual entitlements.

12. Storage Layer

Record Storage is structured in three logical layers:

1. **Raw Layer** — the original transcript and call metadata, stored immutably as received.
2. **Structured Layer** — the extracted fields, their citations, the documentation snippets used, and the rule validation results, stored as a versioned structured record linked to the Call Record ID.
3. **Decision & Audit Layer** — the decision trace (which path was taken and why), plus any human review corrections, stored append-only.

All three layers share the same Call Record ID, allowing a single query to reconstruct the complete lifecycle of a record: what was said, what was retrieved, what was extracted, what was validated, what was decided, and what a human changed.

13. Review Queue

The Review Queue sits between the Decision Engine and the Human Review Screen. Responsibilities: - Holds records routed to Human Review or Escalation in a durable, prioritized queue. - Prioritizes by business signal (e.g., compliance-flagged records surface above simple lowconfidence records). - Supports assignment (round-robin or skill-based routing to reviewers). - Tracks queue age to prevent stale, unreviewed records from silently aging out — records exceeding an SLA threshold are auto-escalated to a supervisor queue.

The queue is a durable store, not an in-memory list, so that reviewer sessions, service restarts, or scaling events never lose a pending record.

14. Dashboard Module

Built on top of the Analytics Module, the Dashboard presents:

- **Volume metrics:** calls processed, auto-save rate, review rate, escalation rate.
- **Quality metrics:** field-level correction frequency, which rules trigger most often, per-agent or per-topic error patterns.
- **Operational metrics:** review queue age/backlog, reviewer throughput.
- **Compliance view:** escalation reasons broken down by regulatory rule category, for audit reporting.

The dashboard also provides business-level operational summaries, including category distribution and sentiment distribution, allowing supervisors and management to monitor call trends and overall customer experience.

The dashboard reads exclusively from Record Storage and the Analytics Module's aggregated views — it never queries the live pipeline directly, keeping reporting load isolated from processing load.

15. Security Considerations

- **Data in transit:** All inter-service communication (Intake → Lookup → Search → Extraction → Validation → Decision → Storage) is encrypted (TLS) between services.
- **Data at rest:** Transcripts and customer context are encrypted at rest; encryption keys are managed centrally and rotated on a defined schedule.
- **PII minimization:** The AI Documentation Engine and Documentation Search Engine receive only the minimum customer context needed (e.g., product entitlements) rather than full customer master records.
- **Access control:** Role-based access separates who can view raw transcripts, who can review/ edit records, and who can access the audit/decision trail; reviewers only see records assigned/ routed to them.
- **Audit immutability:** The Decision & Audit Layer is append-only; corrections are recorded as new entries, never as overwrites, preserving a tamper-evident history.
- **Segregation of duties:** Business rule configuration changes are managed through a separate change-controlled process from day-to-day pipeline operation, preventing silent rule tampering.

16. Error Handling

Failure Point	Handling Strategy
Malformed/unusable transcript	Rejected at Intake with a structured error event; does not enter the pipeline.
Customer Lookup fails or times out	Record proceeds with "Unresolved Identity" flag; forced toward Review/Escalation rather than blocked.
Documentation Search returns no relevant snippets	AI Extraction proceeds but any field requiring policy interpretation is marked "unsupported," forcing Review.
AI Extraction produces low-confidence fields	Never silently accepted — always surfaced to the Decision Engine's thresholds.
Business Rule Validator errors (e.g., missing rule config)	Fails safe: record is routed to Escalation, never to Auto-Save, on validator error.
Storage write failure	Retried with idempotent write (keyed by Call Record ID) to prevent duplicate or partial records.
Review Queue backlog / SLA breach	Automatic re-prioritization and supervisor escalation rather than silent aging.

The guiding principle throughout is **fail toward caution**: any uncertainty or system fault biases the record toward Human Review or Escalation, never toward Auto-Save.

17. Scalability Considerations

- **Event-driven decoupling:** Because each pipeline stage communicates via events rather than direct synchronous calls, stages can be scaled independently based on load (e.g., Documentation Search scaled up during high call volume without needing to scale Human Review capacity in lockstep).
- **Stateless processing services:** Transcript Intake, Customer Lookup, Documentation Search, AI Extraction, and Business Rule Validation are stateless and horizontally scalable; state lives only in the supporting data stores.
- **Queue-based buffering:** Bursty call volumes (e.g., end-of-shift spikes) are absorbed by the event stream and Review Queue rather than causing backpressure on upstream telephony systems.
- **Read/write separation:** Analytics and Dashboard workloads read from replicated/aggregated views of Record Storage, isolating reporting load from live pipeline throughput.

- **Caching of documentation retrieval:** Frequently retrieved policy documents can be cached at the Documentation Search layer (with version/effective-date awareness) to reduce repeated lookup cost at scale.

18. Future Improvements

- **Active learning loop:** Feed Human Review corrections back into periodic model evaluation to continuously reduce the review rate over time without compromising grounding guarantees.
- **Real-time (in-call) assistance:** Extend the same retrieval-and-validation architecture to support live agent guidance during the call rather than only after-call processing.
- **Multi-language transcript support:** Extend Documentation Search and AI Extraction to operate across additional languages with language-matched knowledge base retrieval.
- **Explainability enhancements:** Richer visualizations in the Review Screen showing exactly which transcript span and which documentation snippet drove each field and each rule outcome.
- **Dynamic rule authoring interface:** A controlled UI for business/compliance teams to author and version Business Rule Validator logic without engineering involvement, while preserving change control and testability.
- **Anomaly detection on decision trends:** Automated alerting when auto-save rates, escalation rates, or correction patterns shift significantly, as an early signal of model drift, rule misconfiguration, or upstream data quality issues.

Summary: How the Architecture Prevents Hallucination and Enforces Correctness

This design achieves its core guarantees through **structural constraints**, not model instructions:

- **Grounding** is enforced because the AI Documentation Engine's output schema requires a transcript span citation per field; fields without one are marked unsupported by construction, not by asking the model to "be careful."
- **Retrieval before generation** is enforced because the Documentation Search Engine runs as an earlier, separate pipeline stage — the AI Extraction step architecturally cannot run before documentation is fetched, and it receives only the retrieved snippets, not open-ended access to a knowledge base.
- **Business rules in code, not prompts** are enforced because the Business Rule Validator is a distinct deterministic service the AI output must pass through; a rule cannot be "talked around" by phrasing, since it is evaluated as code against structured data, independent of how the model phrased its extraction.

- **Auto-Save / Review / Escalation decisions** are enforced by an explicit, versioned Decision Engine that combines confidence scores and rule outcomes — never a judgment made inside the generative step itself.

The result is a pipeline where the AI component contributes language understanding and extraction, while every consequential decision is made — and can be audited — outside of it.